

RV UNIVERSITY

School of Computer Science and Engineering

WEEKLY PROGRESS DIARY (LOGBOOK)

External AI/ML Internship at CodSoft (Review 1 Evaluation)

Student Name: Sreeram Lagisetty — USN: 1RUA24CSE0219 — Duration: May 18, 2026 – June 19, 2026

Internship Compliance Metrics

Total Weeks Logged: 5 Weeks

Total Days: 25 Working Days

Total Hours Logged: 79 Hours

Target Subsystems: Chatbot, Recommender, Tic-Tac-Toe AI

Week 1: Requirements Analysis, Environment Setup, and Subsystem Planning

Date	Hours	Technical Tasks Performed
Week 1, Day 1 (May 18, 2026)	3	Cloned the workspace repository containing the three git submodules. Analyzed repository architecture, configured git paths, initialized a Python 3.10 virtual environment, and mapped out dependency requirements for the three subsystems.
Week 1, Day 2 (May 19, 2026)	3	Set up local Python development environment. Installed required framework packages including FastAPI, Uvicorn, SQLite3, and Pytest. Structured the directory layout for backend services and static frontends, ensuring modular Separation of Concerns.
Week 1, Day 3 (May 20, 2026)	3	Studied the requirements and mathematical foundations of all three modules. Formulated specifications for regular-expression NLP matching, collaborative filtering via matrix factorization, content-based recommendation via cosine similarity, and minimax game theory.
Week 1, Day 4 (May 21, 2026)	3	Designed the database schema in SQLite for the Movie Recommendation System. Configured tables for movies, users, and ratings. Coded a database adapter in database.py and initialized it with mock movie and user ratings data for early functional testing.
Week 1, Day 5 (May 22, 2026)	3	Analyzed database connection methods and prepared schema diagrams for the recommendation model. Formulated the mathematical integration strategy for aligning content-based user profile vectors with FunkSVD latent factors.

Weekly Review (Week 1)

- **Technical Assessment:** Evaluated the database design and validated SQLite relational schema indexing.
- **Implementation Status:** Verified unique index constraints on user/movie ID columns to optimize query speeds.

Week 2: Rule-Based Chatbot Subsystem Development

Date	Hours	Technical Tasks Performed
Week 2, Day 1 (May 25, 2026)	3	Commenced coding the Rule-Based Chatbot. Created intents.json to catalog conversational intents, regular expression patterns with capture groups, and text response templates. Coded the text normalizer in chatbot.py to sanitize user input by handling case folding, stripping punctuation, and cleaning whitespace.
Week 2, Day 2 (May 26, 2026)	4	Programmed the Intent Classifier and Slot Filler in Python. Wrote logic to dynamically compile regular expressions from intents.json, match input sequences, and extract variables from named capture groups (such as names and locations) to fill context slots.
Week 2, Day 3 (May 27, 2026)	3	Designed and integrated the stateful Session Manager within chatbot.py. Created a context dictionary mapped to distinct user session IDs, allowing the chatbot to maintain conversational state and context-aware responses (such as addressing users by their captured names).
Week 2, Day 4 (May 28, 2026)	3	Developed the web interface for the chatbot using HTML5, vanilla CSS, and JavaScript. Designed a responsive chat widget with glassmorphism styling. Ported the regex classification and slot-filling logic into app.js to run client-side.
Week 2, Day 5 (May 29, 2026)	3	Wrote unit tests in test_chatbot.py using the unittest module. Verified correct behavior of normalizer functions, pattern classifiers, and session state retention. Added randomized fallback response arrays to intents.json to handle unmatched inputs.

Weekly Review (Week 2)

- **Technical Assessment:** Tested regex patterns and checked name capture group slot-filling functions in chatbot.py.
- **Implementation Status:** Configured randomized fallback response arrays in intents.json to handle unmatched inputs.

Week 3: Movie Recommendation System Subsystem Development

Date	Hours	Technical Tasks Performed
Week 3, Day 1 (June 01, 2026)	3	Started the Recommendation System codebase. Programmed the Content-Based Filtering algorithm in recommender.py. Coded the logic to compute user profile vectors from historical movie ratings, centering the rating scores around a 2.5 baseline.
Week 3, Day 2 (June 02, 2026)	3	Developed the Cosine Similarity metric to compare user profile vectors against movie genre multi-hot vectors. Coded the mathematical formula utilizing dot products divided by L2 norms. Verified similarity outputs using sample vector pairs.
Week 3, Day 3 (June 03, 2026)	4	Programmed the Collaborative Filtering engine using FunkSVD matrix factorization. Coded the Stochastic Gradient Descent (SGD) training loop to decompose the user-movie rating matrix into user-latent (P) and movie-latent (Q) matrices, applying L_2 regularization to control overfitting.
Week 3, Day 4 (June 04, 2026)	3	Created the hybrid recommendation blender in recommender.py. Blended SVD scores and Content-Based scores using a weighting coefficient of 0.6. Built FastAPI API endpoints in backend/main.py to serve recommendations to the frontend client.
Week 3, Day 5 (June 05, 2026)	3	Designed the CineMatch Dashboard frontend using HTML, CSS, and vanilla JS. Programmed dynamic movie card rendering and direct user rating submittals that trigger model updates. Configured training testing modules in tests/test_recommender.py to validate SVD model weights convergence.

Weekly Review (Week 3)

- **Technical Assessment:** Scrutinized the FunkSVD rating prediction scaling and rating convergence constraints.
- **Implementation Status:** Developed min-max normalization logic in recommender.py to scale SVD scores to 0-to-1 bounds.

Week 4: Unbeatable Tic-Tac-Toe AI Subsystem Development

Date	Hours	Technical Tasks Performed
Week 4, Day 1 (June 08, 2026)	3	Started development of the Tic-Tac-Toe AI. Defined the board representation using a 9-element array and set up game endpoints. Coded the frontend board layout and interactive grid styling, complete with state animations.
Week 4, Day 2 (June 09, 2026)	3	Coded the core recursive Minimax decision tree algorithm in minimax.py. Implemented depth-discounted utilities, assigning +10 minus depth for AI wins (rewarding faster victories) and -10 plus depth for human wins (delaying losses).
Week 4, Day 3 (June 10, 2026)	3	Added Alpha-Beta Pruning optimization to the Minimax search tree. Implemented pruning bounds checks where recursion halts if the beta value drops below or matches the alpha value. Tested the performance improvement in state evaluations.
Week 4, Day 4 (June 11, 2026)	4	Integrated a Fail-Soft Transposition Cache in the minimax solver. Developed key hashing to store evaluated board states. Programmed depth-neutral normalization for cached scores, preventing depth-discounted values from polluting other branches.
Week 4, Day 5 (June 12, 2026)	3	Created verify_unbeatable.py to simulate all 255,168 possible game state sequences, validating that the AI never loses a game. Optimized initial moves lookup by seeding the transposition cache with empty-board and single-move evaluations.

Weekly Review (Week 4)

- **Technical Assessment:** Audited minimax depth-discounted values and early game response latency.
- **Implementation Status:** Pre-seeded the transposition cache with empty-board states to achieve 0ms initial moves computation.

Week 5: Subsystem Integration, Testing, and Code Profiling

Date	Hours	Technical Tasks Performed
Week 5, Day 1 (June 15, 2026)	3	Commenced system integration. Combined the Rule-Based Chatbot, CineMatch Movie Recommendation System, and Tic-Tac-Toe AI into a unified monorepo. Consolidated project dependencies into a root requirements.txt and organized static routing.
Week 5, Day 2 (June 16, 2026)	3	Wrote comprehensive integration and end-to-end regression tests using Pytest. Created assertions for chatbot intent classification, SQLite database CRUD operations, SVD recommender training loops, and Minimax solver accuracy.
Week 5, Day 3 (June 17, 2026)	3	Conducted performance profiling across the unified application. Used Python cProfile to measure API response times and verify transposition cache hits. Analyzed SQLite connection pool size and optimized DB query paths.
Week 5, Day 4 (June 18, 2026)	4	Finalized technical documentation and structural project reports. Drafted architectural system flows, outlined mathematical formulations (SVD updates, Minimax equations), and organized test coverage summaries.
Week 5, Day 5 (June 19, 2026)	3	Finalized the repository integration, cleaned up temporary debug logs, and verified the final build. Documented the system architectures, mathematical formulas (such as SGD parameter updates and Minimax recursion bounds), and testing coverage in the project README.

Weekly Review (Week 5)

- **Technical Assessment:** Inspected codebase documentation, docstring mathematical derivations, and integration builds.
- **Implementation Status:** Documented FunkSVD gradient update steps and Tic-Tac-Toe state space proofs in docstrings.